

C# - Phần 10.2: LINQ Select()

Lý thuyết - Câu hỏi lý thuyết - Bài tập - Đáp án - Lưu ý

Mục tiêu: Hiểu Select() dùng để lấy hoặc biến đổi dữ liệu từ mỗi phần tử, phân biệt Select() với Where(), hiểu kiểu kết quả, cách ghép Where() + Select(), và cách chuyển Product thành ProductSummary.

PHẦN A - LÝ THUYẾT

1. Select() dùng để làm gì?

Nếu Where() trả lời câu hỏi 'phần tử nào được giữ lại?', thì Select() trả lời câu hỏi 'từ mỗi phần tử, ta muốn lấy hoặc biến đổi thành dữ liệu gì?'.

```
var productNames =  
    products.Select(  
        product => product.Name);
```

Ở đây mỗi Product được biến đổi thành một string là Name của nó.

2. Bản chất của Lambda trong Select()

```
product => product.Name
```

Với List<Product>, biến product có kiểu Product và product.Name có kiểu string.

```
Product -> string
```

Select() áp dụng Lambda lên từng phần tử của collection đầu vào để tạo ra phần tử đầu ra.

3. Select() với nhiều kiểu kết quả

```
products.Select(product => product.Name)  
→ IEnumerable<string>  
  
products.Select(product => product.Price)  
→ IEnumerable<decimal>  
  
products.Select(  
    product =>  
        new ProductSummary(  
            product.Name,  
            product.Price))  
→ IEnumerable<ProductSummary>
```

Ghi nhớ: Kiểu kết quả của Select() phụ thuộc vào kiểu dữ liệu mà Lambda trả về.

4. Select() biến đổi dữ liệu

Select() không chỉ lấy property; nó còn có thể tạo chuỗi mới, tính toán hoặc tạo object mới.

```
var descriptions =  
    products.Select(  
        product =>  
            $"{product.Id} - {product.Name}");  
  
var newPrices =  
    products.Select(  
        product => product.Price * 1.1m);
```

5. Select() tạo object mới

```
List<ProductSummary> summaries =  
    products  
        .Select(  
            product =>
```

```

        new ProductSummary(
            product.Name,
            product.Price))
    .ToList();

```

Lambda ở đây có dạng Product -> ProductSummary.

Về tư duy, mỗi Product được truyền vào Lambda; Lambda lấy Name và Price rồi tạo một ProductSummary mới.

6. Hiểu Select() bằng foreach truyền thống

```

List<ProductSummary> summaries =
    new List<ProductSummary>();

foreach (Product product in products)
{
    ProductSummary summary =
        new ProductSummary(
            product.Name,
            product.Price);

    summaries.Add(summary);
}

```

LINQ Select() giúp viết gọn cùng ý tưởng biến đổi mỗi phần tử thành một phần tử mới.

7. Where() và Select() khác nhau thế nào?

```

Where()
→ giữ PHẦN TỬ nào?

Select()
→ lấy / biến đổi DỮ LIỆU gì từ mỗi phần tử?

products
    .Where(product => product.Price >= 1000)
    .Select(product => product.Name);

```

Where() vẫn giữ kiểu Product nhưng giảm số lượng phần tử. Select() có thể đổi Product thành string, decimal, ProductSummary, ...

8. Ghép Where() + Select() + ToList()

```

List<string> names =
    products
        .Where(
            product => product.Price >= 1000)
        .Select(
            product => product.Name)
        .ToList();

List<Product>
    ↓ Where
IEnumerable<Product>
    ↓ Select
IEnumerable<string>
    ↓ ToList
List<string>

```

9. var với Select()

```

var names =
    products.Select(
        product => product.Name);
// IEnumerable<string>

var namesList =
    products
        .Select(

```

```

        product => product.Name)
        .ToList();
// List<string>

```

var chỉ giúp compiler tự suy luận kiểu từ biểu thức bên phải; var không quyết định kiểu.

10. Khi nào cần foreach?

Nếu kết quả cuối cùng vẫn là một collection như IEnumerable<T> hoặc List<T>, muốn hiển thị tất cả phần tử thì foreach là cách tự nhiên.

```

List<string> names =
    products
        .Select(product => product.Name)
        .ToList();

foreach (string name in names)
{
    Console.WriteLine(name);
}

```

Nếu sau này truy vấn chỉ trả về một giá trị duy nhất, ví dụ sau FirstOrDefault(), thì không cần foreach.

Điểm dễ nhầm: Một Lambda trả một giá trị cho mỗi phần tử không có nghĩa toàn bộ Select() chỉ trả một giá trị. Nếu có 5 Product thì có thể tạo ra 5 giá trị đầu ra.

11. Select() không sửa collection gốc

Select() tạo ra một chuỗi kết quả mới; nó không tự thay đổi các object trong List gốc nếu Lambda chỉ đọc dữ liệu để tạo giá trị mới.

PHẦN B - CÂU HỎI LÝ THUYẾT

1. Select() dùng để làm gì?
2. Where() và Select() khác nhau ở điểm nào?
3. Với Lambda product => product.Name, product có kiểu gì và Lambda trả kiểu gì?
4. Nếu var result = products.Select(product => product.Name); thì result có dạng kiểu gì?
5. Nếu thêm .ToList() thì kiểu kết quả thay đổi như thế nào?
6. Vì sao Select() có thể biến Product thành string, decimal hoặc ProductSummary?
7. Với products.Where(...).Select(...).ToList(), hãy mô tả dữ liệu thay đổi qua từng bước.

PHẦN C - BÀI TẬP 10.2

Tiếp tục dùng class Product từ bài 10.1:

```

public class Product
{
    public string Id { get; }
    public string Name { get; }
    public decimal Price { get; }

    // Constructor + validation
}

```

Tạo thêm:

```

public class ProductSummary
{
    public string Name { get; }
    public decimal Price { get; }

    // Constructor + validation
}

```

Dữ liệu:

```
List<Product> products =
    new List<Product>
    {
        new Product("P01", "Mouse", 300),
        new Product("P02", "Keyboard", 700),
        new Product("P03", "Monitor", 2500),
        new Product("P04", "Laptop", 15000),
        new Product("P05", "USB", 150)
    };
```

- Dùng Select() lấy tất cả Name thành List<string>.
- Dùng Select() lấy tất cả Price thành List<decimal>.
- Tạo List<string> dạng: P01 - Mouse, P02 - Keyboard, ...
- Dùng Where() + Select() để lấy tên Product có Price >= 1000.
- Dùng Where() + Select() để lấy giá Product có Price < 1000.
- Dùng Select() để chuyển List<Product> thành List<ProductSummary>.
- Dùng foreach để hiển thị từng kết quả.
- Không sửa trực tiếp danh sách products.

PHẦN D - ĐÁP ÁN

1. ProductSummary.cs

```
public class ProductSummary
{
    public string Name { get; }
    public decimal Price { get; }

    public ProductSummary(
        string name,
        decimal price)
    {
        if (string.IsNullOrEmpty(name))
        {
            throw new ArgumentException(
                "Product summary name cannot be null or whitespace.",
                nameof(name));
        }

        if (price <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(price),
                "Price must be greater than 0.");
        }

        Name = name.Trim();
        Price = price;
    }

    public override string ToString()
    {
        return $"Name: {Name}, Price: {Price}";
    }
}
```

2. Program.cs

```
public class Program
{
    public static void Main(string[] args)
    {
        List<Product> products =
            new List<Product>
```

```

        {
            new Product("P01", "Mouse", 300),
            new Product("P02", "Keyboard", 700),
            new Product("P03", "Monitor", 2500),
            new Product("P04", "Laptop", 15000),
            new Product("P05", "USB", 150)
        };

// 1. Select Name
List<string> productNames =
    products
        .Select(product => product.Name)
        .ToList();

// 2. Select Price
List<decimal> productPrices =
    products
        .Select(product => product.Price)
        .ToList();

// 3. P01 - Mouse
List<string> productLabels =
    products
        .Select(
            product =>
                $"{product.Id} - {product.Name}")
        .ToList();

// 4. Where + Select Name
List<string> expensiveNames =
    products
        .Where(
            product => product.Price >= 1000)
        .Select(
            product => product.Name)
        .ToList();

// 5. Where + Select Price
List<decimal> cheapPrices =
    products
        .Where(
            product => product.Price < 1000)
        .Select(
            product => product.Price)
        .ToList();

// 6. Product -> ProductSummary
List<ProductSummary> summaries =
    products
        .Select(
            product =>
                new ProductSummary(
                    product.Name,
                    product.Price))
        .ToList();

foreach (string name in productNames)
{
    Console.WriteLine(name);
}

foreach (decimal price in productPrices)
{
    Console.WriteLine(price);
}

foreach (string label in productLabels)

```

```

    {
        Console.WriteLine(label);
    }

    foreach (string name in expensiveNames)
    {
        Console.WriteLine(name);
    }

    foreach (decimal price in cheapPrices)
    {
        Console.WriteLine(price);
    }

    foreach (ProductSummary summary in summaries)
    {
        Console.WriteLine(summary);
    }
}

```

3. Đáp án 7 câu lý thuyết

1. Select() dùng để làm gì?

Select() dùng để lấy hoặc biến đổi dữ liệu từ mỗi phần tử của collection thành một giá trị/phần tử mới.

2. Where() và Select() khác nhau ở điểm nào?

Where() lọc phần tử theo điều kiện; Select() lấy hoặc biến đổi dữ liệu từ mỗi phần tử và có thể làm thay đổi kiểu dữ liệu của kết quả.

3. product => product.Name có kiểu gì?

product có kiểu Product; Lambda trả về string, nên có dạng Product -> string.

4. products.Select(product => product.Name) trả dạng gì?

Kết quả có dạng IEnumerable<string>.

5. Thêm ToList() thì sao?

IEnumerable<string> được chuyển thành List<string>.

6. Vì sao Select() biến Product thành nhiều kiểu được?

Vì kiểu đầu ra của Select() phụ thuộc vào giá trị mà Lambda trả về. Lambda có thể trả string, decimal, ProductSummary hoặc kiểu khác phù hợp.

7. Where().Select().ToList() thay đổi dữ liệu thế nào?

Bắt đầu từ collection gốc; Where() lọc và vẫn giữ kiểu phần tử ban đầu; Select() biến mỗi phần tử còn lại thành kiểu dữ liệu mong muốn; ToList() chuyển chuỗi kết quả thành List của kiểu đầu ra.

PHẦN E - ĐÁNH GIÁ BÀI LÀM VÀ LƯU Ý

- Select Name: đúng.
- Select Price: đúng.
- Where + Select Name với Price >= 1000: đúng.
- Product -> ProductSummary: đúng và thể hiện tốt bản chất của Select().
- Bài 'P01 - Mouse' ban đầu lấy Name + Price nên sai yêu cầu; đáp án đúng dùng Id + Name.
- Bài Price < 1000 ban đầu Select Name nên sai yêu cầu; đáp án đúng phải Select Price và kết quả là List<decimal>.
- ProductSummary validation nên dùng nameof(price), không dùng nameof(Price).
- Phần code thể hiện Select() tốt hơn phần diễn đạt lý thuyết ban đầu.

1. Lưu ý quan trọng nhất: đừng nhầm Select với Where

Where()
→ phần tử NÀO được giữ lại?

```
Select()  
→ từ mỗi phần tử lấy / biến đổi thành DỮ LIỆU GÌ?
```

Từ 'Select' dễ khiến người học nghĩ là 'chọn phần tử'. Trong LINQ, cách nhớ an toàn hơn là: Select = projection / biến đổi dữ liệu.

2. Kiểu kết quả phụ thuộc Lambda

```
product => product.Name  
→ Product -> string  
→ IEnumerable<string>  
  
product => product.Price  
→ Product -> decimal  
→ IEnumerable<decimal>  
  
product => new ProductSummary(...)  
→ Product -> ProductSummary  
→ IEnumerable<ProductSummary>
```

3. foreach và kết quả collection

Nếu Select() tạo ra nhiều phần tử và kết quả vẫn là IEnumerable<T> hoặc List<T>, muốn hiển thị tất cả thì dùng foreach. Nếu truy vấn sau này chỉ trả về một giá trị duy nhất thì không cần foreach.

4. Tổng kết bài 10.2

```
Select()  
→ áp dụng Lambda lên từng phần tử  
  
Mỗi phần tử đầu vào  
→ một giá trị/phần tử đầu ra  
  
Where()  
→ lọc  
  
Select()  
→ biến đổi  
  
ToList()  
→ chuyển IEnumerable<T> thành List<T>  
  
Product -> string  
Product -> decimal  
Product -> ProductSummary
```

Cầu nối sang 10.3: Sau Where() và Select(), bước tiếp theo là học cách lấy một phần tử cụ thể bằng First(), FirstOrDefault(), Single() và SingleOrDefault().